

Chat em Tempo Real via Sockets TCP

Trabalho de Redes de Computadores

1. Descrição da Aplicação

A aplicação desenvolvida é um **sistema de chat em tempo real** com múltiplos usuários, seguindo o modelo cliente-servidor. O servidor aceita conexões simultâneas de vários clientes e retransmite cada mensagem recebida para todos os demais participantes da sala (broadcast). A comunicação é feita por meio de **sockets TCP**, garantindo entrega ordenada e confiável dos dados.

A escolha do TCP se justifica pelo fato de que em um chat é fundamental que todas as mensagens cheguem íntegras e na ordem correta. O protocolo UDP, embora mais rápido, não oferece essas garantias.

2. Arquitetura e Funcionamento

2.1 Modelo Cliente-Servidor

O sistema é composto por dois módulos independentes: **servidor.py** e **cliente.py**. O servidor fica aguardando conexões em uma porta fixa (padrão: 5000). Cada novo cliente que se conecta recebe uma thread dedicada, permitindo que múltiplas conversas aconteçam simultaneamente sem bloqueio.

2.2 Fluxo de uma mensagem

- Cliente digita uma mensagem e a envia via TCP ao servidor.
- O servidor recebe a mensagem na thread do cliente correspondente.
- A função `broadcast()` percorre a lista de clientes conectados.
- A mensagem é retransmitida para todos, exceto o remetente original.
- O remetente também recebe o eco da própria mensagem (confirmação).

2.3 Protocolo de comunicação

Os dados são transmitidos como strings codificadas em UTF-8, suportando caracteres especiais e acentuação. O tamanho máximo de cada pacote é de 1024 bytes. O comando **/sair** encerra a conexão de forma limpa, notificando os demais usuários.

3. Código-Fonte

3.1 servidor.py

```
# servidor.py - Servidor de Chat TCP
```

```

import socket, threading

HOST, PORT = '0.0.0.0', 5000
clientes = []
lock = threading.Lock()

def broadcast(mensagem, remetente_conn=None):
    with lock:
        for conn, addr, apelido in clientes:
            if conn != remetente_conn:
                try:
                    conn.sendall(mensagem.encode('utf-8'))
                except:
                    pass

def handle_client(conn, addr):
    conn.sendall("Digite seu apelido: ".encode('utf-8'))
    apelido = conn.recv(1024).decode('utf-8').strip() or str(addr)
    with lock:
        clientes.append((conn, addr, apelido))
    broadcast(f"[Servidor] {apelido} entrou no chat!", conn)
    conn.sendall(f"[Servidor] Bem-vindo, {apelido}! /sair para sair.\n".encode())
    try:
        while True:
            dados = conn.recv(1024)
            if not dados: break
            msg = dados.decode('utf-8').strip()
            if msg.lower() == '/sair': break
            texto = f"[{apelido}]: {msg}"
            broadcast(texto, conn)
            conn.sendall((texto + "\n").encode('utf-8'))
    except ConnectionResetError:
        pass
    finally:
        with lock:
            clientes[:] = [(c,a,n) for c,a,n in clientes if c != conn]
        broadcast(f"[Servidor] {apelido} saiu do chat.")
        conn.close()

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor.bind((HOST, PORT))
servidor.listen(10)
print(f"Servidor TCP em {HOST}:{PORT}")
while True:
    conn, addr = servidor.accept()
    threading.Thread(target=handle_client, args=(conn, addr), daemon=True).start()

```

3.2 cliente.py

```

# cliente.py - Cliente de Chat TCP
import socket, threading, sys

HOST, PORT = '127.0.0.1', 5000

def receber(conn):
    while True:
        try:
            dados = conn.recv(1024)
            if not dados: break

```

```

        print(dados.decode('utf-8'), end='', flush=True)
    except:
        break

if len(sys.argv) >= 3:
    HOST, PORT = sys.argv[1], int(sys.argv[2])

conn = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
conn.connect((HOST, PORT))
threading.Thread(target=receber, args=(conn, ), daemon=True).start()

while True:
    entrada = input()
    if entrada.strip().lower() == '/sair':
        conn.sendall('/sair'.encode()); break
    if entrada.strip():
        conn.sendall(entrada.encode('utf-8'))
conn.close()

```

4. Manual do Usuário

4.1 Pré-requisitos

É necessário ter **Python 3.7 ou superior** instalado. Nenhuma biblioteca externa é requerida — o sistema utiliza apenas módulos da biblioteca padrão do Python (*socket* e *threading*).

4.2 Como iniciar o Servidor

Abra um terminal e execute:

```
python servidor.py
```

O servidor ficará escutando na porta 5000 de todas as interfaces de rede da máquina. Mantenha este terminal aberto durante toda a sessão de chat.

4.3 Como conectar um Cliente

Em cada computador (ou terminal) que deseja participar do chat, execute:

```
python cliente.py
```

Para conectar a um servidor em outra máquina, informe o IP e a porta:

```
python cliente.py 192.168.1.100 5000
```

Ao conectar, o sistema pedirá que o usuário informe seu **apelido**. Após isso, basta digitar as mensagens e pressionar Enter para enviá-las.

4.4 Comandos disponíveis

- **/sair** — Encerra a conexão com o servidor e fecha o cliente.

4.5 Teste local com múltiplos clientes

Para testar na mesma máquina, abra **3 terminais**:

- Terminal 1: execute **python servidor.py**
- Terminal 2: execute **python cliente.py** e insira o apelido "Alice"

- Terminal 3: execute **python cliente.py** e insira o apelido "Bob"

Ao digitar uma mensagem em qualquer terminal-cliente, ela aparecerá nos demais terminais em tempo real.

5. Detalhes Técnicos

5.1 Threads e concorrência

Cada cliente aceito pelo servidor gera uma thread independente (*threading.Thread*). A lista de clientes ativos é protegida por um *threading.Lock*, evitando condições de corrida durante leitura e escrita concorrentes.

5.2 Tratamento de desconexões

Quando um cliente se desconecta abruptamente (fechamento de janela, queda de rede, etc.), a exceção *ConnectionResetError* é capturada na thread correspondente. O cliente é removido da lista e os demais são notificados automaticamente.

5.3 Parâmetros configuráveis

- **HOST** (servidor): endereço de escuta — padrão '0.0.0.0' (todas as interfaces)
- **PORT**: porta TCP — padrão 5000 (configurável nos dois arquivos)
- **Backlog** do listen: 10 conexões pendentes simultâneas
- **Buffer de recepção**: 1024 bytes por pacote

6. Estrutura de Arquivos do Projeto

```
chat_sockets/  
■■■ servidor.py      # Servidor TCP multithread  
■■■ cliente.py      # Cliente TCP com thread de recepção  
■■■ README.md       # Link para o vídeo-manual (YouTube)
```

7. Referências

- Python Docs – socket: <https://docs.python.org/3/library/socket.html>
- Python Docs – threading: <https://docs.python.org/3/library/threading.html>
- DataCamp – Socket Programming in Python:
<https://www.datacamp.com/tutorial/a-complete-guide-to-socket-programming-in-python>
- IBM Docs – How sockets work:
https://www.ibm.com/docs/ssw_ibm_i_73/rzab6/howdosockets.htm
- Board Infinity – Socket in Computer Network:
<https://www.boardinfinity.com/blog/socket-in-computer-network/>